

La solution : état de la session

Référence : [Documentation ADK - état de la session](#)

Extrait de la documentation ADK :

"Conceptuellement, `session.state` est une collection (dictionnaire ou mappage) contenant des paires clé-valeur. L'état de la session est conçu pour recueillir les informations dont l'agent doit se souvenir ou qu'il doit pouvoir suivre afin que la conversation en cours soit pertinente :

- **Personnaliser l'interaction** : mémoriser les préférences utilisateur mentionnées précédemment (par exemple, `'user_preference_theme': 'dark'`)
- **Suivre l'avancement des tâches** : garder une trace des étapes dans un processus multitour (par exemple, `'booking_step': 'confirm_payment'`)
- **Accumuler des informations** : constituer des listes ou des résumés (par exemple, `'shopping_cart_items': ['book', 'pen']`)
- **Prendre des décisions éclairées** : stocker les flags ou les valeurs influençant la réponse suivante (par exemple, `'user_is_authenticated': True`)."

Qu'est-ce que l'état de la session ?

L'état de la session est un **dictionnaire Python** accessible dans l'attribut `session.state`. Vous pouvez le considérer comme des variables que votre agent stocke et que votre code peut vérifier :

```
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

# Créer un agent avec output_key pour enregistrer la réponse
agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Extrais le nom de l'utilisateur. Renvoie UNIQUEMENT le nom.",
    output_key="user_name" # Enregistre la réponse dans state["user_name"]
)

# Configurer la session et l'exécuteur
session_service = InMemorySessionService()
session = session_service.create_session(app_name="my_app", user_id="user1")
runner = Runner(agent=agent, app_name="my_app", session_service=session_service)

# Exécuter l'agent
result = runner.run(user_id="user1", session_id=session.id, new_message="My name is Alex")
```

```
# Accéder à l'état en dehors de l'agent
print(session.state) # {'user_name': 'Alex'}
print(session.state.get("user_name")) # 'Alex'

# Désormais, votre code peut prendre des décisions :
if session.state.get("user_name"):
    print("✅ L'utilisateur a fourni son nom.")
```

Que permet l'état ?

- ✅ **Accès programmatique** : votre code peut lire des valeurs exactes avec `session.state.get("key")`.
- ✅ **Enregistrement automatique** : utilisez `output_key` pour enregistrer les réponses de l'agent sans codage manuel.
- ✅ **Prise de décision** : vérifiez les valeurs d'état pour prendre des décisions programmatiques.
- ✅ **Persistance des données** : les valeurs restent accessibles d'un tour de conversation à l'autre.

Concepts fondamentaux

1. Comparaison entre l'état et l'historique de la conversation

Les deux sont à la disposition de vos agents, mais ils ont des objectifs différents :

Fonctionnalité	Historique de la conversation	État de la session
Description	Messages (utilisateur + agent)	Dictionnaire de paires clé-valeur
Lecture par le LLM	✅ Oui (automatique)	✅ Oui (avec la création de modèles <code>{var}</code> , partie 2)
Vérification par le code	❌ Non	✅ Oui (<code>session.state.get("key")</code>)
Utilisation	Contexte pour le LLM	Valeurs exactes pour votre code
Accès	Automatique (envoyé au modèle)	Programmatique (<code>session.state</code>)

Comparaison visuelle :

```
graph TB
    subgraph "Conversation History"
        CH1[Text messages<br/>User and Agent]
        CH2[LLM reads ✅]
        CH3[Your code CANNOT access ❌]
    end
```

```

end

subgraph "Session State"
    SS1[Dictionary<br/>key-value pairs]
    SS2[LLM can read ✓]
    SS3[Your code CAN access ✓]
end

style CH2 fill:#e1f5ff
style CH3 fill:#ffebee
style SS2 fill:#e1f5ff
style SS3 fill:#eefbee

```

Quand utiliser l'état ?

- ✓ Lorsque votre code doit vérifier des valeurs exactes (`if state.get("name"):`)
- ✓ Lorsque vous prenez des décisions de routage de manière programmatique
- ✓ Lorsque vous suivez la progression des conversations ou des informations utilisateur
- ✗ NE PAS utiliser pour fournir uniquement du contexte au LLM (l'historique de la conversation le fait déjà)

2. Accéder à l'état avec `session.state`

Référence : [Documentation ADK - Accès à l'état](#)

Après avoir exécuté votre agent, vous accédez à l'état avec le dictionnaire `session.state` :

```

from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Créer une session
session_service = InMemorySessionService()
session = session_service.create_session(app_name="app", user_id="user1")

# Exécuter l'agent
runner = Runner(agent=my_agent, app_name="app", session_service=session_service)
result = runner.run(
    user_id="user1",
    session_id=session.id,
    new_message=Content(parts=[Part(text="Hello")])
)

# Accéder à l'état APRÈS l'exécution de l'agent
print(session.state) # Dictionnaire d'état complet
name = session.state.get("user_name", "Guest") # Lecture sécurisée avec la valeur par défaut
count = session.state.get("count", 0) # Affiche 0 si la clé n'existe pas

```

Points essentiels :

- L'état est accessible sous forme de dictionnaire : `session.state`.
 - Utilisez `.get(key, default)` pour une lecture sécurisée (affiche la valeur par défaut si la clé n'existe pas).
 - L'état est conservé d'un tour à l'autre au cours de la même session.
 - Votre code peut vérifier les valeurs d'état pour prendre des décisions programmatiques.
-

3. Enregistrer des réponses avec `output_key`

Référence : [Documentation ADK - output_key](#)

Extrait de la documentation ADK :

"`output_key` (facultatif) : fournissez une clé de chaîne. Si cette clé est définie, le contenu textuel de la *dernière* réponse de l'agent sera automatiquement enregistré dans le dictionnaire d'état de la session sous cette clé. Cela permet de transmettre les résultats entre agents ou étapes d'un workflow."

La manière la plus simple d'enregistrer la réponse d'un agent dans l'état consiste à utiliser `output_key` :

```
from google.adk.agents import LlmAgent

agent = LlmAgent(
    model='gemini-2.5-flash',
    instruction="Extrais le thème principal. Renvoie UNIQUEMENT le thème.",
    output_key="topic" # Enregistre automatiquement la réponse dans state["topic"]
)

# Une fois l'agent exécuté, state["topic"] contient le texte de la réponse
# Aucun code manuel n'est nécessaire
```

Fonctionnement :

1. L'agent génère une réponse (par exemple, "informatique quantique").
2. Le framework l'enregistre automatiquement dans `state["topic"] = "quantum computing"`.
3. Vous pouvez y accéder avec `session.state.get("topic")`.

À quel moment l'utiliser ?

-  Lorsque vous souhaitez enregistrer la réponse de l'agent pour l'utiliser ultérieurement
 -  Lorsque vous avez besoin d'un accès programmatique à ce que l'agent a dit
 -  Lorsque vous créez des applications devant suivre les informations extraites
 -  NE PAS utiliser pour les réponses que vous n'aurez pas besoin de consulter ultérieurement
-

Exemple pratique

Créer un agent d'extraction de noms

Objectifs de l'atelier : créer un agent simple qui extrait le nom d'un utilisateur et l'enregistre dans l'état, illustrant comment l'état permet un accès programmatique.

Étape 1 : Créer le projet

```
adk create name_extractor
cd name_extractor
```

Étape 2 : Rédiger l'agent

Remplacez le contenu de `agent.py` par :

```
"""
Agent d'extraction du nom : illustre les principes de base de l'état de la session
Montre comment utiliser output_key pour enregistrer des données et y accéder avec
session.state.

Référence : https://google.github.io/adk-docs/sessions/state.md
"""

from google.adk.agents import LlmAgent

# Agent unique qui extrait et enregistre le nom
root_agent = LlmAgent(
    model='gemini-2.5-flash',
    name='name_extractor',
    instruction="Extrais du message le nom de la personne. Affiche UNIQUEMENT le nom, rien d'autre.",
    output_key="user_name" # Enregistre la réponse dans state["user_name"]
)
```

Tutoriel du code :

Lignes 10 à 16 : agent d'extraction du nom

- Utilise `output_key="user_name"` pour enregistrer automatiquement la réponse.
 - Une fois l'exécution effectuée par l'agent, `state["user_name"]` contient le nom extrait.
 - Aucun code de gestion manuelle de l'état n'est nécessaire.
-

Étape 3 : Tester avec l'exécuteur

Au lieu d'utiliser `adk web`, utilisons directement `Runner` pour visualiser l'accès à l'état :

Créez un fichier nommé `test_state.py` :

```
"""
Tester le script pour visualiser directement l'accès à l'état.
Exécuter avec : python test_state.py
"""

from agent import root_agent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService
from google.genai.types import Content, Part

# Configurer la session et l'exécuteur
session_service = InMemorySessionService()
session = session_service.create_session(
    app_name="name_extractor_app",
    user_id="test_user",
    session_id="test_session"
)

runner = Runner(
    agent=root_agent,
    app_name="name_extractor_app",
    session_service=session_service
)

# Tester : extraire le nom
user_message = Content(parts=[Part(text="Hi, my name is Alex Johnson")])

print("=== Exécution de l'agent en cours ===")
result = runner.run(
    user_id="test_user",
    session_id="test_session",
    new_message=user_message
)

# Afficher la dernière réponse
for event in result:
    if event.is_final_response():
        print(f"\nAgent response: {event.content.parts[0].text}")

# Accéder à l'état de manière programmatique
print(f"\n=== État après exécution ===")
print(f"Full state: {session.state}")
print(f"Extracted name: {session.state.get('user_name')}")

# Votre code peut désormais prendre des décisions en fonction de l'état
if session.state.get("user_name"):
    print("✅ Le nom a bien été extrait et enregistré")
```

```

else:
    print("❌ Échec de l'extraction du nom")

# Tester l'accès dans les tours suivants
print("\n=== Simuler le deuxième tour ===")
result2 = runner.run(
    user_id="test_user",
    session_id="test_session",
    new_message=Content(parts=[Part(text="What's my name?")])
)

for event in result2:
    if event.is_final_response():
        print(f"Agent response: {event.content.parts[0].text}")

print(f"\nState still contains: {session.state.get('user_name')}")
print("✅ L'état persiste d'un tour à l'autre")

```

Exécutez l'agent :

```
python test_state.py
```

Résultat attendu :

```
=== Exécution de l'agent en cours ===
```

```
Réponse de l'agent : Alex Johnson
```

```
=== État après exécution ===
```

```
État complet : {'user_name': 'Alex Johnson'}
```

```
Nom extrait : Alex Johnson
```

```
✅ Le nom a bien été extrait et enregistré
```

```
=== Simulation du deuxième tour ===
```

```
Réponse de l'agent : Tu t'appelles Alex Johnson
```

```
L'état comprend toujours : Alex Johnson
```

```
✅ L'état persiste d'un tour à l'autre
```

Points à noter :

- ✅ `output_key="user_name"` a automatiquement enregistré "Alex Johnson" dans l'état.
- ✅ Votre code peut accéder à `session.state.get("user_name")` de manière programmatique.
- ✅ Vous pouvez prendre des décisions : `if session.state.get("user_name"):`
- ✅ L'état est conservé d'un tour à l'autre au cours de la même session.

Points à retenir

Principes fondamentaux concernant l'état de la session :

- L'**état de la session** est un dictionnaire dans lequel les agents stockent des données auxquelles votre code peut accéder de manière programmatique.
- L'**historique de la conversation** fournit un contexte au LLM, mais votre code ne peut pas le vérifier de manière programmatique.
- Les deux sont disponibles et se complètent (l'historique pour le contexte du LLM, l'état pour le contrôle programmatique).

Comment utiliser l'état ?

- Accédez à l'état avec `session.state` après avoir exécuté les agents avec `Runner`.
- Utilisez `.get(key, default)` pour lire les valeurs d'état de façon sécurisée.
- Utilisez `output_key="key"` pour enregistrer automatiquement les réponses des agents dans l'état.
- L'état est conservé d'un tour à l'autre au cours de la même session.

Quand utiliser l'état ?

-  Lorsque votre code doit vérifier des valeurs exactes de manière programmatique
-  Lorsque vous prenez des décisions de routage basées sur les données de conversation
-  Lorsque vous suivez la progression des conversations ou des informations utilisateur
-  Lorsque vous stockez des informations extraites pour la logique d'application

Schéma courant :

```
# 1. L'agent enregistre dans l'état avec output_key
agent = LlmAgent(output_key="result")

# 2. Exécuter l'agent
runner.run(...)

# 3. Accéder à l'état de manière programmatique
if session.state.get("result"):
    # Prendre des décisions en fonction de l'état
    pass
```

Partie suivante : vous allez apprendre à utiliser la création de modèles `{var}` pour injecter des valeurs d'état dans les instructions, permettant à l'agent d'avoir un comportement dynamique.